IE 620 Presentation
AlphaStock: Investment Strategy using Deep Reinforcement Attention Networks

Dhairya Kantawala

IIT Bombay

08/04/2026

1. **Overview**
   - Why did I choose this paper?
   - How did I read this paper?

2. **Main Results and their Significance**
   - MDP model
   - Function Approximation
   - Policy Gradient (REINFORCE)
   - Algorithm
   - Convergence

3. **What I want to do**
   - Implementation
   - Extension to the paper

Why did I choose this paper?

- Quantitative finance is a growing field, the main objective has always been to make as much money as possible (yearly returns are a popular choice for $J(\theta)$)

Why did I choose this paper?

- Quantitative finance is a growing field, the main objective has always been to make as much money as possible (yearly returns are a popular choice for $J(\theta)$)
- But then I came across this paper that tries to minimize risk (essentially maximizing the Sharpe ratio).

Why did I choose this paper?

- Quantitative finance is a growing field, the main objective has always been to make as much money as possible (yearly returns are a popular choice for $J(\theta)$)
- But then I came across this paper that tries to minimize risk (essentially maximizing the Sharpe ratio).
- What we just saw last few weeks with the war is a good example of why risk management is important, and maybe it is time to focus and make portfolios that manage risk better than trying to seek good returns and losing all your capital.

Why did I choose this paper?

- Quantitative finance is a growing field, the main objective has always been to make as much money as possible (yearly returns are a popular choice for $J(\theta)$)
- But then I came across this paper that tries to minimize risk (essentially maximizing the Sharpe ratio).
- What we just saw last few weeks with the war is a good example of why risk management is important, and maybe it is time to focus and make portfolios that manage risk better than trying to seek good returns and losing all your capital.
- So why this paper?

Why did I choose this paper?

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

Why did I choose this paper?

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

$$\mu(\theta) = \frac{1}{T}\sum_{t=1}^{T} R_t \quad , \quad \sigma(\theta) = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(R_t - \mu(\theta))^2}$$

Why did I choose this paper?

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

$$\mu(\theta) = \frac{1}{T}\sum_{t=1}^{T} R_t \quad , \quad \sigma(\theta) = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(R_t - \mu(\theta))^2}$$

The above is just a fancy way of writing the Sharpe ratio. The more the ratio, the more robust and less risky is our strategy.

How did I read this paper?

I will not waste much time here but the paper "How to Read a Paper, S. Keshav" actually quite helped me.

How did I read this paper?

I will not waste much time here but the paper "How to Read a Paper, S. Keshav" actually quite helped me.

- First Pass: quick scan to get a bird's-eye view
- Second Pass: read the paper with greater care, but ignore proofs
- Third Pass: virtually re-implement the paper (The second part of my project, still left)

MDP Model

We model this as a Markov Decision Process:

$$(\mathcal{S}, \mathcal{A}, P, R)$$

MDP Model

We model this as a Markov Decision Process:

$$(\mathcal{S}, \mathcal{A}, P, R)$$

State:

$$s_t = \{X_t^{(1)}, X_t^{(2)}, \ldots, X_t^{(N)}\}, \quad X_t^{(i)} \in \mathbb{R}^{K \times d}$$

This is basically the last K timesteps of features for all stocks.

MDP Model

We model this as a Markov Decision Process:

$$(\mathcal{S}, \mathcal{A}, P, R)$$

State:

$$s_t = \{X_t^{(1)}, X_t^{(2)}, \ldots, X_t^{(N)}\}, \quad X_t^{(i)} \in \mathbb{R}^{K \times d}$$

This is basically the last K timesteps of features for all stocks.

Action:

$$a_t = b_t = (b_t^{(1)}, \ldots, b_t^{(N)})$$

Portfolio weights:

$$\sum_i b_t^{+(i)} = 1, \quad \sum_i b_t^{-(i)} = 1$$

(long on top stocks, short on bottom ones, zero-investment portfolio)

MDP Model

We model this as a Markov Decision Process:

$$(\mathcal{S}, \mathcal{A}, P, R)$$

State:

$$s_t = \{X_t^{(1)}, X_t^{(2)}, \ldots, X_t^{(N)}\}, \quad X_t^{(i)} \in \mathbb{R}^{K \times d}$$

This is basically the last K timesteps of features for all stocks.

Action:

$$a_t = b_t = (b_t^{(1)}, \ldots, b_t^{(N)})$$

Portfolio weights:

$$\sum_i b_t^{+(i)} = 1, \quad \sum_i b_t^{-(i)} = 1$$

(long on top stocks, short on bottom ones, zero-investment portfolio)

Transition:

$$s_{t+1} \sim P(\cdot \mid s_t, a_t)$$

Market evolves independently of us (we assume we are small).

Market evolves independently of us (we assume we are small).
Reward:

$$R_t = \sum_i b_t^{+(i)} z_t^{(i)} - \sum_i b_t^{-(i)} z_t^{(i)}$$

$$z_t^{(i)} = \frac{p_{t+1}^{(i)}}{p_t^{(i)}}$$

So we are not predicting price, just trying to ensure:

longs outperform shorts

Market evolves independently of us (we assume we are small).

Reward:

$$R_t = \sum_i b_t^{+(i)} z_t^{(i)} - \sum_i b_t^{-(i)} z_t^{(i)}$$

$$z_t^{(i)} = \frac{p_{t+1}^{(i)}}{p_t^{(i)}}$$

So we are not predicting price, just trying to ensure:

longs outperform shorts

Policy:

$$a_t = \mu_\theta(s_t)$$

Market evolves independently of us (we assume we are small).

Reward:

$$R_t = \sum_i b_t^{+(i)} z_t^{(i)} - \sum_i b_t^{-(i)} z_t^{(i)}$$

$$z_t^{(i)} = \frac{p_{t+1}^{(i)}}{p_t^{(i)}}$$

So we are not predicting price, just trying to ensure:

longs outperform shorts

Policy:

$$a_t = \mu_\theta(s_t)$$

The key challenge is that we don't have $P(\cdot|s_t, a_t)$, so we will use a policy-based RL method (REINFORCE) with Non-Linear Function Approximation to solve this problem.

Features to Representation

For each stock i, we start with:

$$X_t^{(i)} = \{x_{t-K+1}^{(i)}, \ldots, x_t^{(i)}\}, \quad x_k^{(i)} \in \mathbb{R}^d$$

(last K timesteps of features)

Features to Representation

For each stock i, we start with:

$$X_t^{(i)} = \{x_{t-K+1}^{(i)}, \ldots, x_t^{(i)}\}, \quad x_k^{(i)} \in \mathbb{R}^d$$

(last K timesteps of features)

LSTM Encoder:

$$h_k^{(i)} = \mathrm{LSTM}_\theta(h_{k-1}^{(i)}, x_k^{(i)}), \; h_K^{(i)} \text{ captures sequential dependence}$$

Features to Representation

For each stock i, we start with:

$$X_t^{(i)} = \{x_{t-K+1}^{(i)}, \ldots, x_t^{(i)}\}, \quad x_k^{(i)} \in \mathbb{R}^d$$

(last K timesteps of features)

LSTM Encoder:

$$h_k^{(i)} = \mathrm{LSTM}_\theta(h_{k-1}^{(i)}, x_k^{(i)}), \; h_K^{(i)} \text{ captures sequential dependence}$$

History Attention:

$$r^{(i)} = \sum_{k=1}^{K} \alpha_k^{(i)} h_k^{(i)}$$

$$\alpha_k^{(i)} = \frac{\exp(e_k)}{\sum_{k'} \exp(e_{k'})}, \quad e_k = w_h^\top \tanh(W_1 h_k + W_2 h_K)$$

Features to Representation

For each stock i, we start with:

$$X_t^{(i)} = \{x_{t-K+1}^{(i)}, \ldots, x_t^{(i)}\}, \quad x_k^{(i)} \in \mathbb{R}^d$$

(last K timesteps of features)

LSTM Encoder:

$$h_k^{(i)} = \mathrm{LSTM}_\theta(h_{k-1}^{(i)}, x_k^{(i)}), \; h_K^{(i)} \text{ captures sequential dependence}$$

History Attention:

$$r^{(i)} = \sum_{k=1}^{K} \alpha_k^{(i)} h_k^{(i)}$$

$$\alpha_k^{(i)} = \frac{\exp(e_k)}{\sum_{k'} \exp(e_{k'})}, \quad e_k = w_h^\top \tanh(W_1 h_k + W_2 h_K)$$

So we convert raw time-series $\rightarrow$ a single vector $r^{(i)}$ per stock.

Cross-Asset Attention

Now we model relationships across stocks.

Cross-Asset Attention

Now we model relationships across stocks.

Linear Projections:

$$q^{(i)} = W_Q r^{(i)}, \quad k^{(i)} = W_K r^{(i)}, \quad v^{(i)} = W_V r^{(i)}$$

Cross-Asset Attention

Now we model relationships across stocks.

Linear Projections:

$$q^{(i)} = W_Q r^{(i)}, \quad k^{(i)} = W_K r^{(i)}, \quad v^{(i)} = W_V r^{(i)}$$

Attention across stocks:

$$\alpha_{ij} = \frac{\exp\left(q^{(i)\top} k^{(j)} / \sqrt{d_k}\right)}{\sum_{j'} \exp\left(q^{(i)\top} k^{(j')} / \sqrt{d_k}\right)}$$

$$a^{(i)} = \sum_{j=1}^{N} \alpha_{ij} v^{(j)}$$

Cross-Asset Attention

Now we model relationships across stocks.

Linear Projections:

$$q^{(i)} = W_Q r^{(i)}, \quad k^{(i)} = W_K r^{(i)}, \quad v^{(i)} = W_V r^{(i)}$$

Attention across stocks:

$$\alpha_{ij} = \frac{\exp\left(q^{(i)\top} k^{(j)} / \sqrt{d_k}\right)}{\sum_{j'} \exp\left(q^{(i)\top} k^{(j')} / \sqrt{d_k}\right)}$$

$$a^{(i)} = \sum_{j=1}^{N} \alpha_{ij} v^{(j)}$$

Winner score:

$$s^{(i)} = \sigma\left(w_s^\top a^{(i)} + b_s\right)$$

Cross-Asset Attention

Now we model relationships across stocks.

Linear Projections:

$$q^{(i)} = W_Q r^{(i)}, \quad k^{(i)} = W_K r^{(i)}, \quad v^{(i)} = W_V r^{(i)}$$

Attention across stocks:

$$\alpha_{ij} = \frac{\exp\left(q^{(i)\top} k^{(j)} / \sqrt{d_k}\right)}{\sum_{j'} \exp\left(q^{(i)\top} k^{(j')} / \sqrt{d_k}\right)}$$

$$a^{(i)} = \sum_{j=1}^{N} \alpha_{ij} v^{(j)}$$

Winner score:

$$s^{(i)} = \sigma\left(w_s^\top a^{(i)} + b_s\right)$$

So score of stock i depends on **all other stocks** (relative ranking).

Portfolio and Objective

Portfolio construction:

Top G stocks $\rightarrow$ long, bottom G $\rightarrow$ short

$$b^{+(i)} = \frac{e^{s^{(i)}}}{\sum_{j \in \mathcal{G}_+} e^{s^{(j)}}}, \quad b^{-(i)} = \frac{e^{1-s^{(i)}}}{\sum_{j \in \mathcal{G}_-} e^{1-s^{(j)}}}$$

Portfolio and Objective

Portfolio construction:

Top G stocks $\rightarrow$ long, bottom G $\rightarrow$ short

$$b^{+(i)} = \frac{e^{s^{(i)}}}{\sum_{j \in \mathcal{G}_+} e^{s^{(j)}}}, \quad b^{-(i)} = \frac{e^{1-s^{(i)}}}{\sum_{j \in \mathcal{G}_-} e^{1-s^{(j)}}}$$

Return:

$$R_t = \sum_i b_t^{+(i)} z_t^{(i)} - \sum_i b_t^{-(i)} z_t^{(i)}, \quad z_t^{(i)} = \frac{p_{t+1}^{(i)}}{p_t^{(i)}}$$

Portfolio and Objective

Portfolio construction:

Top G stocks $\rightarrow$ long, bottom G $\rightarrow$ short

$$b^{+(i)} = \frac{e^{s^{(i)}}}{\sum_{j \in \mathcal{G}_+} e^{s^{(j)}}}, \quad b^{-(i)} = \frac{e^{1-s^{(i)}}}{\sum_{j \in \mathcal{G}_-} e^{1-s^{(j)}}}$$

Return:

$$R_t = \sum_i b_t^{+(i)} z_t^{(i)} - \sum_i b_t^{-(i)} z_t^{(i)}, \quad z_t^{(i)} = \frac{p_{t+1}^{(i)}}{p_t^{(i)}}$$

Objective (Sharpe ratio):

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

$$\mu = \frac{1}{T} \sum_t R_t, \quad \sigma = \sqrt{\frac{1}{T} \sum_t (R_t - \mu)^2}$$

The Policy

Policy:

$$a_t = \mu_\theta(s_t)$$

Here θ = all parameters of the model:

$$\theta = \{\theta_{\text{LSTM}}, W_Q, W_K, W_V, w_s, b_s\}$$

The Policy

Policy:

$$a_t = \mu_\theta(s_t)$$

Here θ = all parameters of the model:

$$\theta = \{\theta_{\text{LSTM}}, W_Q, W_K, W_V, w_s, b_s\}$$

Pipeline:

$$X_t^{(i)} \to r^{(i)} \to s^{(i)} \to b_t$$

The Policy

Policy:

$$a_t = \mu_\theta(s_t)$$

Here θ = all parameters of the model:

$$\theta = \{\theta_{\text{LSTM}}, W_Q, W_K, W_V, w_s, b_s\}$$

Pipeline:

$$X_t^{(i)} \to r^{(i)} \to s^{(i)} \to b_t$$

So the policy is learning a function:

$$\mu_\theta : s_t \to b_t$$

which outputs portfolio weights directly. There is **no separate critic or value function** , this is a pure policy-based method.

Goal

Goal:

$$\max_{\theta} \; J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

Goal

Goal:

$$\max_{\theta} \; J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

So θ controls:

- feature extraction (LSTM)
- cross-stock interactions (attention)
- final ranking (scores)

Goal

Goal:

$$\max_{\theta} \; J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

So θ controls:

- feature extraction (LSTM)
- cross-stock interactions (attention)
- final ranking (scores)

But how do we compute $\nabla_\theta J(\theta)$? The reward is **not separable** across timesteps , the Sharpe ratio mixes all R_t together. We need to be careful.

Gradient of the Sharpe Objective

We are not maximizing $\sum_t R_t$, instead:

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

$$\mu = \frac{1}{T}\sum_t R_t, \quad \sigma = \sqrt{\frac{1}{T}\sum_t (R_t - \mu)^2}$$

Gradient of the Sharpe Objective

We are not maximizing $\sum_t R_t$, instead:

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

$$\mu = \frac{1}{T}\sum_t R_t, \quad \sigma = \sqrt{\frac{1}{T}\sum_t (R_t - \mu)^2}$$

So reward is **not separable across time**.

Gradient of the Sharpe Objective

We are not maximizing $\sum_t R_t$, instead:

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

$$\mu = \frac{1}{T}\sum_t R_t, \quad \sigma = \sqrt{\frac{1}{T}\sum_t (R_t - \mu)^2}$$

So reward is **not separable across time**.
Gradient of Sharpe (by chain rule):

$$\nabla_\theta J(\theta) = \sum_{t=1}^{T} w_t \, \nabla_\theta R_t$$

where:

$$w_t = \frac{1}{T\sigma} - \frac{\mu}{T\sigma^3}(R_t - \mu)$$

Gradient of the Sharpe Objective

We are not maximizing $\sum_t R_t$, instead:

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

$$\mu = \frac{1}{T}\sum_t R_t, \quad \sigma = \sqrt{\frac{1}{T}\sum_t (R_t - \mu)^2}$$

So reward is **not separable across time**.

Gradient of Sharpe (by chain rule):

$$\nabla_\theta J(\theta) = \sum_{t=1}^{T} w_t \, \nabla_\theta R_t$$

where:

$$w_t = \frac{1}{T\sigma} - \frac{\mu}{T\sigma^3}(R_t - \mu)$$

So each timestep's contribution is **weighted** by how much it deviates from

REINFORCE with Sharpe Ratio

The paper uses a **policy gradient (REINFORCE)** approach. A full trajectory π of T periods is collected, and the Sharpe ratio H_π of that trajectory is used as the scalar reward signal.

REINFORCE with Sharpe Ratio

The paper uses a **policy gradient (REINFORCE)** approach. A full trajectory π of T periods is collected, and the Sharpe ratio H_π of that trajectory is used as the scalar reward signal.

Policy gradient objective:

$$J(\theta) = \int_\pi H_\pi \, \Pr(\pi|\theta) \, d\pi$$

REINFORCE with Sharpe Ratio

The paper uses a **policy gradient (REINFORCE)** approach. A full trajectory π of T periods is collected, and the Sharpe ratio H_π of that trajectory is used as the scalar reward signal.

Policy gradient objective:

$$J(\theta) = \int_\pi H_\pi \, \Pr(\pi|\theta) \, d\pi$$

Gradient estimate over N trajectories:

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{n=1}^{N} \left(H_{\pi_n} \sum_{t=1}^{T} \sum_{i} \nabla_\theta \log \Pr\left(\text{action}_t^{(i)} = 1 \mid X_t^{(n)}, \theta \right) \right)$$

REINFORCE with Sharpe Ratio

The paper uses a **policy gradient (REINFORCE)** approach. A full trajectory π of T periods is collected, and the Sharpe ratio H_π of that trajectory is used as the scalar reward signal.

Policy gradient objective:

$$J(\theta) = \int_\pi H_\pi \, \Pr(\pi|\theta) \, d\pi$$

Gradient estimate over N trajectories:

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{n=1}^{N} \left(H_{\pi_n} \sum_{t=1}^{T} \sum_{i} \nabla_\theta \log \Pr\left(\text{action}_t^{(i)} = 1 \mid X_t^{(n)}, \theta \right) \right)$$

Note: H_{π_n} **integrally weights all timesteps** in the trajectory. The reward is not given to any isolated step but to the whole trajectory , this is what makes it far-sighted.

Market-Beating Threshold

There is one more trick. To ensure the model actually **beats the market** and not just learns a mediocre strategy, the paper introduces a threshold H_0:

Market-Beating Threshold

There is one more trick. To ensure the model actually **beats the market** and not just learns a mediocre strategy, the paper introduces a threshold H_0:

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{n=1}^{N} \left((H_{\pi_n} - H_0) \sum_{t=1}^{T} \sum_{i} \nabla_\theta \log G^{(i)} \left(X_t^{(n)}, \theta \right) \right)$$

Market-Beating Threshold

There is one more trick. To ensure the model actually **beats the market** and not just learns a mediocre strategy, the paper introduces a threshold H_0:

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{n=1}^{N} \left((H_{\pi_n} - H_0) \sum_{t=1}^{T} \sum_{i} \nabla_\theta \log G^{(i)} \left(X_t^{(n)}, \theta \right) \right)$$

where H_0 is the Sharpe ratio of the overall market (e.g. the buy-and-hold benchmark).

Market-Beating Threshold

There is one more trick. To ensure the model actually **beats the market** and not just learns a mediocre strategy, the paper introduces a threshold H_0:

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{n=1}^{N} \left((H_{\pi_n} - H_0) \sum_{t=1}^{T} \sum_{i} \nabla_\theta \log G^{(i)} \left(X_t^{(n)}, \theta \right) \right)$$

where H_0 is the Sharpe ratio of the overall market (e.g. the buy-and-hold benchmark).

- If $H_{\pi_n} > H_0$: gradient ascent encourages this trajectory
- If $H_{\pi_n} < H_0$: gradient **penalizes** this trajectory

Market-Beating Threshold

There is one more trick. To ensure the model actually **beats the market** and not just learns a mediocre strategy, the paper introduces a threshold H_0:

$$\nabla J(\theta) \approx \frac{1}{N} \sum_{n=1}^{N} \left((H_{\pi_n} - H_0) \sum_{t=1}^{T} \sum_{i} \nabla_\theta \log G^{(i)} \left(X_t^{(n)}, \theta \right) \right)$$

where H_0 is the Sharpe ratio of the overall market (e.g. the buy-and-hold benchmark).

- If $H_{\pi_n} > H_0$: gradient ascent encourages this trajectory
- If $H_{\pi_n} < H_0$: gradient **penalizes** this trajectory

This is essentially a REINFORCE baseline trick, but the baseline is the market's Sharpe ratio rather than a learned value function.

Rollout

Goal: learn θ to maximize

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

Rollout

Goal: learn θ to maximize

$$J(\theta) = \frac{\mu(\theta)}{\sigma(\theta)}$$

```
Initialize policy parameters θ
each episode k = 1, 2, ...
  Initialize state s_1
  t = 1 to T
    Compute representations:
    X_t^(i) → r^(i) → s^(i)
    Policy outputs portfolio:
    a_t = μ_θ(s_t)
    Execute trades, observe reward:
    R_t = Σ_i b_t^+(i) z_t^(i) − Σ_i b_t^−(i) z_t^(i)
    Observe next state s_t+1
```

Learning Updates

Compute returns $\{R_1, \ldots, R_T\}$

Compute Sharpe statistics:

$$\mu = \frac{1}{T} \sum_t R_t$$

$$\sigma = \sqrt{\frac{1}{T} \sum_t (R_t - \mu)^2}$$

$$H_\pi = \mu / \sigma$$

Compute per-timestep weights:

$$w_t = \frac{1}{T\sigma} - \frac{\mu}{T\sigma^3}(R_t - \mu)$$

Policy (gradient ascent):

$$\theta \leftarrow \theta + \alpha \left(H_\pi - H_0\right) \sum_t \nabla_\theta \log G^{(i)}(X_t, \theta)$$

Learning Updates

```
Compute returns {R_1, ..., R_T}
Compute Sharpe statistics:
```

$$\mu = \frac{1}{T}\sum_t R_t$$

$$\sigma = \sqrt{\frac{1}{T}\sum_t (R_t - \mu)^2}$$

$$H_\pi = \mu/\sigma$$

```
Compute per-timestep weights:
```

$$w_t = \frac{1}{T\sigma} - \frac{\mu}{T\sigma^3}(R_t - \mu)$$

```
Policy (gradient ascent):
```

$$\theta \leftarrow \theta + \alpha\,(H_\pi - H_0)\sum_t \nabla_\theta \log G^{(i)}(X_t, \theta)$$

So:

- rollout gives a full trajectory
- Sharpe ratio of the trajectory is computed as a scalar reward
- policy gradient update pushes θ to improve Sharpe **above the market**
- **no critic, no TD learning, no value function** , pure REINFORCE

Convergence

We update parameters as:

$$\theta_{k+1} = \theta_k + \alpha_k \hat{g}_k$$

Convergence

We update parameters as:

$$\theta_{k+1} = \theta_k + \alpha_k \hat{g}_k$$

where:

$$\hat{g}_k = \sum_{t=1}^{T} w_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Convergence

We update parameters as:

$$\theta_{k+1} = \theta_k + \alpha_k \hat{g}_k$$

where:

$$\hat{g}_k = \sum_{t=1}^{T} w_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Key property:

$$\mathbb{E}[\hat{g}_k \mid \theta_k] = \nabla J(\theta_k)$$

(unbiased gradient estimate)

Under standard assumptions:

- bounded rewards
- smooth policy π_θ
- step sizes $\sum \alpha_k = \infty$, $\sum \alpha_k^2 < \infty$

Under standard assumptions:

- bounded rewards
- smooth policy π_θ
- step sizes $\sum \alpha_k = \infty$, $\sum \alpha_k^2 < \infty$

we get:

$$\theta_k \to \theta^* \quad \text{such that} \quad \nabla J(\theta^*) = 0$$

(local optimum)

Under standard assumptions:

- bounded rewards
- smooth policy π_θ
- step sizes $\sum \alpha_k = \infty$, $\sum \alpha_k^2 < \infty$

we get:

$$\theta_k \to \theta^* \quad \text{such that} \quad \nabla J(\theta^*) = 0$$

(local optimum)
Note: because this is REINFORCE and not an actor-critic method, convergence can be slower and variance can be higher , but the paper argues this is acceptable for the financial setting where stability matters more than speed.

Implementation

I want to try do the following:

- Get financial data of NIFTY 500 companies from our Institute's Bloomberg Terminal.

Implementation

I want to try do the following:

- Get financial data of NIFTY 500 companies from our Institute's Bloomberg Terminal.
- Clean the Data and then make sure that the features, dimensions and parameter weight dimensions match that of the paper.

Implementation

I want to try do the following:

- Get financial data of NIFTY 500 companies from our Institute's Bloomberg Terminal.
- Clean the Data and then make sure that the features, dimensions and parameter weight dimensions match that of the paper.
- Code the model in Python using relevant libraries and then train the model parameters by algorithm mentioned in the paper.

Implementation

I want to try do the following:

- Get financial data of NIFTY 500 companies from our Institute's Bloomberg Terminal.
- Clean the Data and then make sure that the features, dimensions and parameter weight dimensions match that of the paper.
- Code the model in Python using relevant libraries and then train the model parameters by algorithm mentioned in the paper.
- Test the model on new data, calculate and compare it with other models and paper results.

Extension to the paper

- Instead of LSTM, I want to use a more basic model like a single MLP layer or 1D CNN model as I think LSTM is an overkill and prone to overfitting as too many weights.

Extension to the paper

- Instead of LSTM, I want to use a more basic model like a single MLP layer or 1D CNN model as I think LSTM is an overkill and prone to overfitting as too many weights.
- Instead of the paper's REINFORCE approach, I want to implement an Actor-Critic method as REINFORCE will have more variance and slow learning, the paper itself does not use a critic, so this would be a genuine architectural extension.